AOBP API URL and HTTP Standard

Status: accepted documentation contract. Reviewed 2 November 2026.

1. Introduction

Use one predictable, versioned API convention across the UI, BFF and microservices. HTTP defines method and status semantics; it does not mandate a universal path such as `/bff/session`. The paths below are AOBP conventions built on HTTP semantics, JSON and Problem Details. They become implementation requirements when adopted in an API contract. This repository contains documentation, so changing these examples does not change a running application.

2. Architecture overview

The browser calls same-origin BFF resources using its opaque session cookie. The BFF selects a configured service route and relays the access token in the Authorization header. Each microservice validates the token and enforces RBAC and ABAC. A URL prefix, gateway route or UI capability never grants authorization.

Boundary	Canonical pattern	Example
Browser BFF REST	`/bff/api/v{major}/{resource}`	`/bff/api/v1/session`
Private domain REST	`/api/v{major}/{resource}`	`/api/v1/resources/resource-123`
External partner REST	Partner host plus the domain contract	`https://api.example.com/api/v1/resources/resource-123`
Browser WebSocket	`/bff/ws/v{major}/events`	`wss://aobp.example.com/bff/ws/v1/events`
OIDC login initiation	Framework security route	`/oauth2/authorization/{registrationId}`
OIDC login callback	Exact registered framework route	`/login/oauth2/code/{registrationId}`
OIDC back-channel logout	Dedicated protocol receiver	`/bff/oidc/backchannel-logout`
Management	Private management listener/routes	`/actuator/health/readiness`

Keep version placement consistent within each boundary. `/bff/api/v1` is the browser namespace; `/api/v1` is the domain namespace. Do not mix `/api/v1/bff`, `/bff/v1/api` and unversioned `/bff/session` in a new deployment. Hostnames and ports change by environment, not resource paths. Protocol routes and management endpoints deliberately do not inherit the business API version.

3. Naming principles

1. Use lowercase collection nouns: `resources`, `flights`, `tasks`, `workflow-instances`.

2. Use kebab-case compound path segments: `logout-requests`, `operational-times`. JSON fields and query parameters use lowerCamelCase.

3. Use singular names for actual singleton resources: `session` is the current caller's session; `csrf-token` is the current session-bound token representation. Pluralization is not an absolute rule for every segment.

4. Use opaque resource IDs and `{resourceId}` in OpenAPI templates. IDs are case-sensitive values, so an existing workflow identifier can retain its original case. Do not lowercase identifiers or rename vendor identifiers as if they were path nouns.

5. Keep nesting shallow and model real ownership: `/resources/{resourceId}/approvals`. Use query parameters for validated filters and pagination.

6. Avoid `/getResources`, `/saveData`, Java class names, pod names, technology names and environment names. Generic `microservice-1` examples in this repository represent stable logical routing namespaces; production APIs should use business capabilities such as `flights`, not Deployment names.

7. Use no trailing slash in canonical URLs. Reject ambiguous encodings, traversal, duplicate/conflicting parameters and invalid IDs. Do not automatically redirect unsafe methods to an alternative spelling.

8. Do not put tokens, passwords, session IDs or personal data in URLs. The OIDC callback's short-lived code/state parameters are protocol exceptions: redact them from access logs, tracing and analytics.

9. Major URL versions represent breaking contracts. Compatible additions remain in v1; breaking changes require coexistence, a migration window and published retirement dates.

4. BFF security endpoint catalogue

Method and canonical URL	Purpose	Success	Protection
`GET /bff/api/v1/session`	Read display identity and UI capabilities	200 JSON; 401 if unauthenticated	Session cookie; no OAuth tokens in the response
`GET /bff/api/v1/csrf-token`	Obtain a token/header name for session-bound CSRF protection	200 JSON	Same origin; bounded anonymous login/bootstrap session if required by implementation
`DELETE /bff/api/v1/session`	End the current application session only	204 with no body	CSRF validation when an active session exists; expire cookie and remove authorized-client state
`POST /bff/api/v1/logout-requests`	Process logout and prepare an approved SSO end-session navigation when configured	200 JSON	Session and CSRF; server controls logout scope and redirect allowlist

For local DELETE logout, an already absent session returns 204 without creating a new session; infrastructure failures are not reported as successful invalidation. A repeated POST must safely handle an

already invalidated session without resurrecting credentials. It can return a completed result without a redirect when no valid SSO logout context remains. Do not promise identity-provider logout completion merely because the BFF session was deleted.

All session, CSRF and logout responses use `Cache-Control: no-store`. Never return access, refresh or ID tokens, raw session references, provider client credentials or encryption material. No browser-facing `/refresh-token` API is required: refresh is owned by the BFF and coordinated per authorized-client record across replicas.

Example session response, a purpose-specific JSON representation rather than a mandatory universal success envelope:

```
{
  "user": { "id": "user-123", "displayName": "Example Operator" },
  "capabilities": ["RESOURCE.READ"],
  "expiresAt": "2026-11-02T12:30:00Z"
}
```

`capabilities` guide UI rendering only; each service checks permissions independently. Publish the exact fields and whether `expiresAt` means idle or absolute expiry in OpenAPI. Do not expose a misleading fixed expiry when activity extends idle lifetime.

Example CSRF response:

```
{ "headerName": "X-CSRF-TOKEN", "token": "opaque-csrf-value" }
```

Example logout response when corporate SSO logout is enabled:

```
{
  "status": "APPLICATION_SESSION_ENDED",
  "redirectUrl": "https://identity.example.com/approved-end-session-url"
}
```

The URL is an illustrative provider result, not a constructed Keycloak endpoint. The BFF builds it using configured/discovered provider metadata and an exact allowlisted return URI. React performs a top-level navigation; it does not fetch the identity provider's logout page. Do not accept an arbitrary client-provided URL or log the returned URL when it contains sensitive protocol values. If SSO logout is disabled or context is absent, omit `redirectUrl` and render the signed-out UI.

5. Business APIs and explicit relay mappings

Browser method/path	Private target	Authorization owner
`GET /bff/api/v1/microservice-1/resources/{resourceId}`	Microservice 1: `GET /api/v1/resources/{resourceId}`	Microservice 1: JWT, RESOURCE.READ and resource scope
`PATCH /bff/api/v1/microservice-1/resources/{resourceId}`	Microservice 1: `PATCH /api/v1/resources/{resourceId}`	Microservice 1: JWT, RESOURCE.UPDATE and resource scope
`GET /bff/api/v1/microservice-2/records/{recordId}`	Microservice 2: `GET /api/v1/records/{recordId}`	Microservice 2: JWT, RECORD.READ and resource scope
`GET /bff/api/v1/flights/{flightId}`	Flight service: `GET /api/v1/flights/{flightId}`	Flight service: JWT, flight read permission and resource scope

The generic and flight rows are different documented use cases, not routes to enable automatically. Select a fixed destination from the method/resource route registry. Do not derive a hostname from a caller's path segment. Security resources under `/bff/api/v1` terminate in the BFF; they must never fall through into the business proxy.

Forward an access token only to an allowlisted destination and intended audience. Strip caller-supplied Authorization/identity headers on the browser boundary and add trusted outbound credentials server-side. Use bounded timeouts and payload sizes. Do not transparently replay a mutation following refresh or a transport error unless its idempotency contract makes the replay safe.

Existing workflow runtime adapter

The step-by-step workflow guide uses these public resource routes while preserving the private runtime's existing contract:

Public BFF operation	Existing private runtime target
`POST /bff/api/v1/runtime/workflows/{workflowId}/instances`	`POST /api/runtime/workflows/{workflowId}/start`
`GET /bff/api/v1/runtime/instances/{instanceId}/tasks`	`GET /api/runtime/instances/{instanceId}/tasks`
`GET /bff/api/v1/runtime/instances/{instanceId}/history`	`GET /api/runtime/instances/{instanceId}/history`
`GET /bff/api/v1/runtime/instances/{instanceId}/variables`	`GET /api/runtime/instances/{instanceId}/variables`
`POST /bff/api/v1/runtime/tasks/{taskId}/completions`	`POST /api/runtime/tasks/{taskId}/complete`

These are required adapter mappings, not proof the runtime or BFF already implements them. Preserve the documented payload/response schemas and explicitly test translation. New owned runtime APIs should use `/api/v1/...`; a vendor contract does not change merely because a browser API has been renamed.

6. HTTP behaviour and response conventions

Operation	Method	Normal success
Read a resource/collection	GET	200; documented conditional GET may return 304
Create a persistent resource	POST collection	201 plus Location in the caller-visible namespace
Start asynchronous work	POST request resource	202 plus authorized status URI
Process a synchronous command without a stored request resource	POST	200 result, or 204 if no result
Replace a documented representation	PUT	200 or 204; creation only if explicitly supported
Update permitted fields	PATCH	200 or 204; ETag/If-Match where concurrency matters
Delete a resource	DELETE	204, or another explicitly documented deletion result

GET/HEAD must not perform business mutations or logout. A CSRF bootstrap can establish security state without being a business mutation. PUT/DELETE are idempotent in intended effect, not necessarily identical response codes. POST/PATCH retries require an explicit contract. An Idempotency-Key feature must define authenticated actor/operation scope, request digest, persistence duration and concurrent duplicate handling; an arbitrary header alone is insufficient.

JSON uses UTF-8 `application/json`. Use purpose-specific objects for individual resources and the platform's documented collection representation for lists; do not wrap 204 responses. Use cursor pagination (`limit`, `cursor`) and allowlisted filters/sort keys. Generate cursors server-side and reapply authorization on every page. Do not leak private hosts in Location, pagination links or error instances; explicitly map downstream URLs to public BFF resources.

Failures use RFC 9457 `application/problem+json`, with the real HTTP status. Existing stable platform business error codes can extend the Problem Details object. Example:

```
{
  "type": "https://aobp.example.com/problems/access-denied",
  "title": "Access denied",
  "status": 403,
  "detail": "You cannot perform this operation on the requested resource.",
  "instance": "/bff/api/v1/microservice-1/resources/resource-123",
  "code": "AOBP_ACCESS_DENIED",
  "traceId": "4bf92f3577b34da6a3ce929d0e0e4736"
}
```

Serve documentation at published problem-type URLs. Never expose stack traces, SQL, tokens, internal hosts or sensitive policy details. Use 400 for malformed requests, 401 for missing/invalid authentication, 403 for an authenticated denial, 404 for absent or deliberately concealed resources, 405 plus Allow for an unsupported method, 409 for state conflict, 412 for failed If-Match, 415 for unsupported media, 422 for documented semantic validation, 429 for throttling, and 503 when mandatory policy/store dependencies prevent a safe decision. Bearer-protected service 401s carry the applicable WWW-Authenticate challenge; browser session APIs return JSON 401 rather than an interactive login redirect. Clients do not infer

authentication scheme solely from the path.

7. Framework, protocol and infrastructure exceptions

Keep Spring's `/oauth2/authorization/{registrationId}` and `/login/oauth2/code/{registrationId}` security routes unless there is an explicit customization requirement. They are login protocol entry/callback routes, not ordinary versioned JSON resources. Register exact callbacks and align Spring security matchers, edge routing and IdP client settings. Do not implement a cosmetic callback alias without updating all three.

Back-channel logout remains a dedicated signed-token receiver at `/bff/oidc/backchannel-logout`. It is exempt only from browser session/CSRF checks and authenticates the OIDC logout token. Provider authorization, token, revocation and end-session endpoints come from trusted issuer metadata/provider configuration, not this naming convention. SCIM and other standards-based integration endpoints retain their own protocol conventions.

WebSocket is an upgrade/protocol endpoint, not a JSON REST operation. Validate session and exact Origin at upgrade, and enforce authorization on messages/subscriptions. Protect all management endpoints on a separate private management boundary; do not proxy Actuator through the general BFF route. Unknown API routes must return an API 404/405, never the React SPA HTML fallback.

8. Migration and step-by-step verification

1. Publish the OpenAPI contracts and exact method/path mappings. Inventory UI calls, Spring matchers, gateway/Ingress rules, WAF rules, CSRF exceptions, monitoring route labels and partner consumers.

2. Replace the old `/bff/session`, `/bff/csrf`, `/bff/logout` paths with the canonical catalogue. Upgrade `/bff/api/runtime/...` and `/bff/ws` variants using the mappings above. Update UI and BFF together or use a documented coexistence window.

3. If legacy paths are temporarily supported, use explicit internal dispatch to the same secured handler, with matching method/auth/CSRF controls and a retirement date. Do not redirect POST/PATCH/DELETE bodies to new paths. The old examples in this migration paragraph are historical mappings, not valid new contracts.

4. Keep OIDC registration/callback routes unchanged; verify exact redirects still work after the route updates. Do not change private vendor APIs without their owner's contract.

5. From a browser, request `GET /bff/api/v1/session` before login: expect JSON 401, no OAuth tokens and no HTML. Complete top-level SSO login, repeat the request and verify 200/no-store.

6. Fetch `GET /bff/api/v1/csrf-token`, then perform an allowed mutation with cookie and the returned header. Missing/invalid CSRF must return 403 without a backend mutation.

7. Read a known allowed resource; then test same-role users outside the resource scope. The service must require both RBAC and ABAC. Wrong issuer/audience/expired tokens must fail at the service even in a controlled direct-network test.

8. Test `DELETE /bff/api/v1/session`: expect 204 with an empty body and an expired cookie; session GET must then return 401. Repeat deletion without a session: expect 204. Test the POST logout-request flow separately with a new authenticated session; only a server-approved SSO URL may be returned.

9. Verify old retired paths, unknown methods, encoded traversal, arbitrary destination hosts and direct browser `/api/**` bypass attempts fail. API errors must not become 200 HTML pages.

10. Verify canonical route templates in logs/traces, no credentials in URLs, public Location/link rewriting, all workflow adapter mappings, socket upgrade/reconnect and replica-independent session handling.

Acceptance evidence must include contract tests against an actual implementation. Renaming examples or regenerating PDFs is not runtime verification.

9. References

- RFC 9110: HTTP Semantics: methods, status codes and HTTP behaviour.

- RFC 9457: Problem Details for HTTP APIs: standardized error representation.

- Spring Security OAuth2 Login advanced configuration: framework authorization and callback routes.

Version placement, resource names, pluralization and pagination choices are platform conventions; they are not an IETF-prescribed URL template.